



SMARTCART OUTPUT EXPLANATION & SYSTEM MANUAL

1. Project Introduction

SmartCart is a full-stack electronic commerce shopping web platform developed using modern multi-tiered client-server architecture. The system combines a responsive Single Page Application (SPA) frontend engineered with **React.js (v16+)**, **HTML5**, **CSS3**, **JavaScript**, and **Axios**, with a robust backend service constructed using **Java 21**, **Spring Boot 3.3**, **Spring Data JPA**, and **Maven**. All persistent application data—including user accounts, product catalogs, categories, and customer orders—is securely stored in a relational **MySQL Database**.

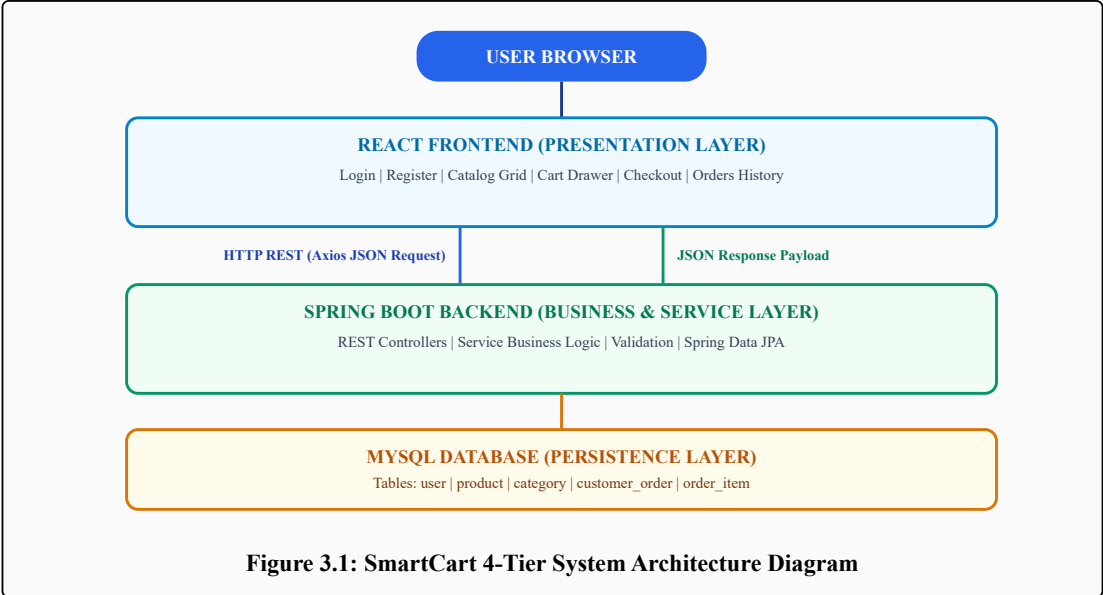
2. Project Objective

The core objectives of the SmartCart platform include:

- **Decoupled Architecture:** Establish a clean operational boundary between the React presentation client and the Spring Boot RESTful microservices backend.
- **Dynamic Shopping User Experience:** Provide customers with instant real-time product keyword searching, category filtering, and price sorting (Low-to-High, High-to-Low, A-Z).
- **State-Persistent Shopping Cart:** Implement a slide-over cart drawer using Context API and local storage state persistence.
- **Order & Checkout Flow:** Capture recipient shipping information, support payment method choices, and persist order records cleanly into relational database tables.
- **Administrative Order Control:** Provide store administrators with real-time customer order management and status update controls (/admin/orders).

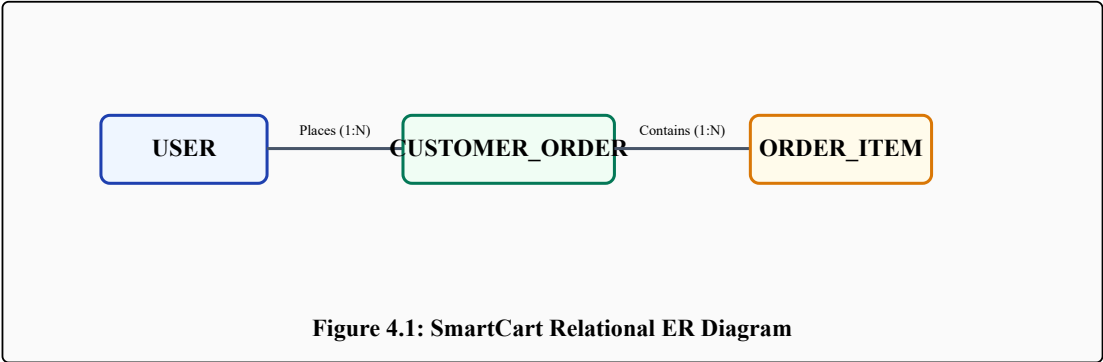
3. System Architecture

The SmartCart architecture follows a strict 4-tier model comprising the Presentation Layer, API Integration Layer, Service Business Layer, and Database Persistence Layer.



4. Entity-Relationship (ER) Diagram

The ER diagram maps the relational data entities and structural constraints governing database interactions:



5. Database Description

The MySQL relational database schema comprises five primary tables and associated sequence tables:

Table 5.1: Database Tables Summary & Relational Foreign Keys

Table Name	Primary Key	Foreign Key(s)	Functional Description
user	username (VARCHAR)	None	Stores registered user accounts, passwords, and admin privileges.
category	id (INT)	None	Organizes items into logical store categories (e.g. Men Shirts, T-Shirts).
product	id (INT)	category_id → category.id	Stores product catalog details, pricing, images, and category linkage.
customer_order	id (BIGINT)	username → user.username	Stores order headers, delivery addresses, status, and order totals.

Table Name	Primary Key	Foreign Key(s)	Functional Description
order_item	id (BIGINT)	order_id → customer_order.id	Stores individual line items, product IDs, quantities, and sub-prices.

6. Backend Implementation

The Spring Boot backend utilizes standard multi-layer encapsulation:

- **Controllers (e.g. UserController, ProductResource, OrderController):** Intercept HTTP REST requests, parse JSON request payloads, enforce cross-origin CORS rules, and return standard ResponseEntity HTTP responses.
- **Services (e.g. UserService, ProductService, OrderService):** Implement transactional business logic, calculate order subtotals, auto-generate timestamps, and handle exception scenarios.
- **Repository Layer (e.g. UserRepo, ProductRepo, OrderRepository):** Extend JpaRepository interfaces to automate object-relational database mapping and SQL query execution against MySQL.
- **REST APIs:** Standardized endpoints (e.g. GET /api/products/all, POST /api/orders/create, GET /api/orders/user/{username}) support asynchronous JSON communication.

7. Frontend Implementation

The React client interface provides a dynamic Single Page Application (SPA):

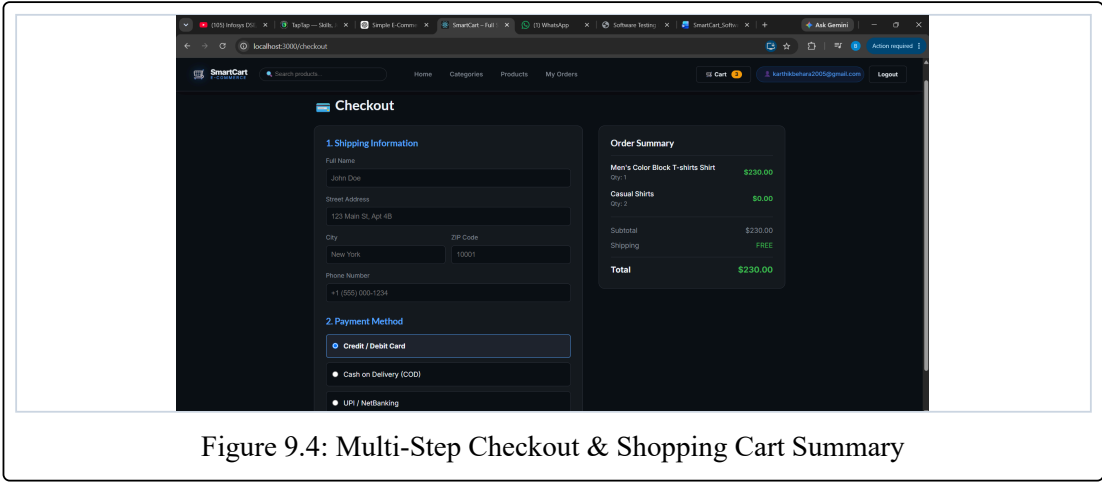
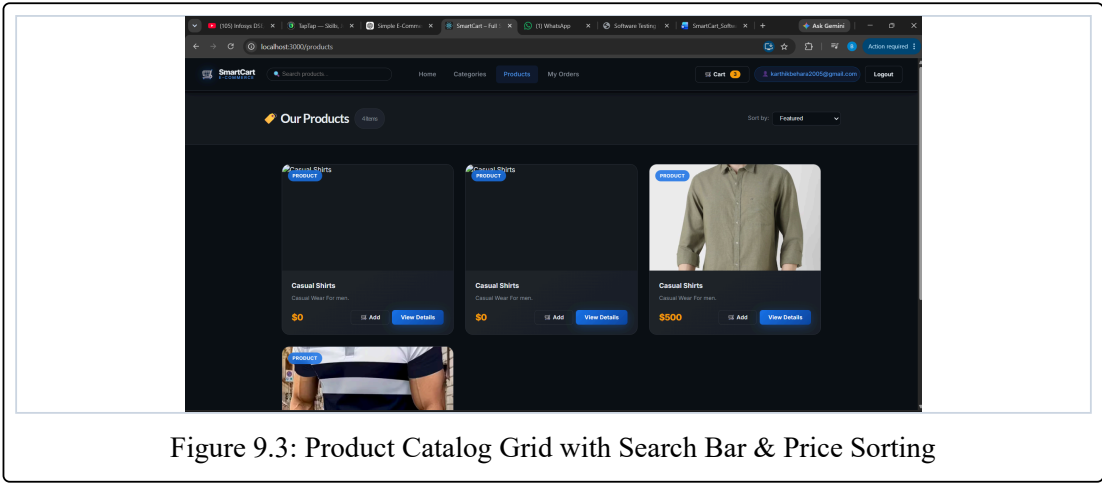
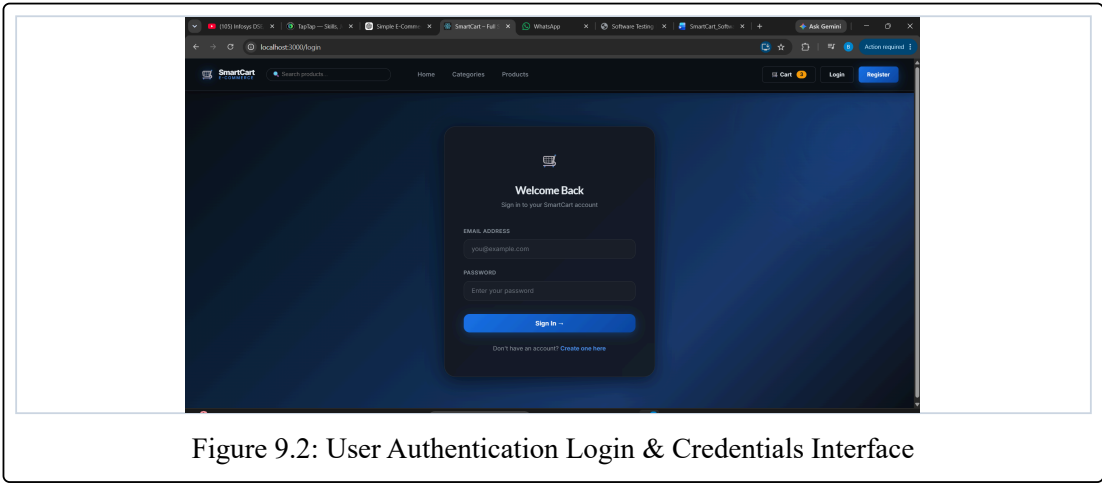
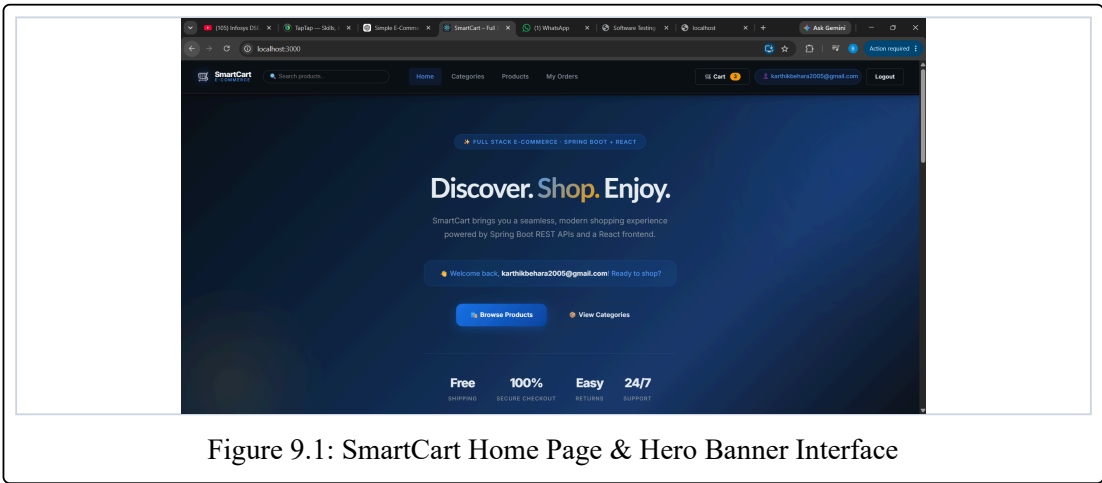
- **React Components:** Functional component architecture including Navbar.js, Products.js, CartModal.js, Checkout.js, and Orders.js.
- **Routing:** Client-side navigation managed via React Router DOM v5 without full page reloads.
- **Axios HTTP Client:** Centralized REST connector module (src/config/connector.js) executing asynchronous HTTP requests to Spring Boot endpoints on port 8080.

8. Application Workflow

The complete end-to-end user transaction workflow proceeds as follows:

1. **Authentication:** Customer registers or logs in via /login. User credentials are validated by Spring Boot and saved in client localStorage.
2. **Catalog Exploration:** Customer browses items under /products, filters by category, sorts by price, or types keywords into the navbar search input.
3. **Cart Management:** Clicking "Add to Cart" updates the global Context API cart drawer, allowing live quantity adjustments and subtotal calculation.
4. **Checkout & Order Placement:** Customer enters shipping address and selects payment method on /checkout, dispatching an Axios POST request to /api/orders/create.
5. **Persistence & Receipt:** Spring Boot saves the order record in MySQL database tables and returns HTTP 200 OK. The customer views their receipt under /orders.

9. Output Screenshots



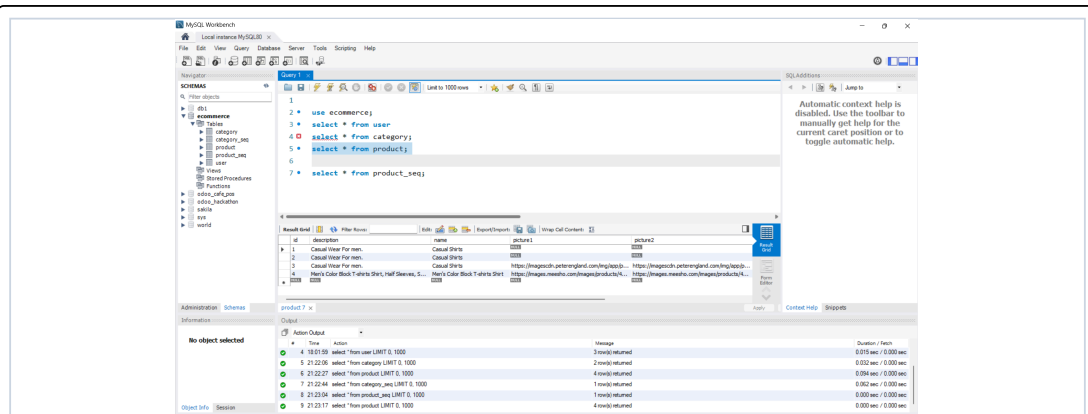


Figure 9.5: MySQL Workbench Database Schema & Query Results

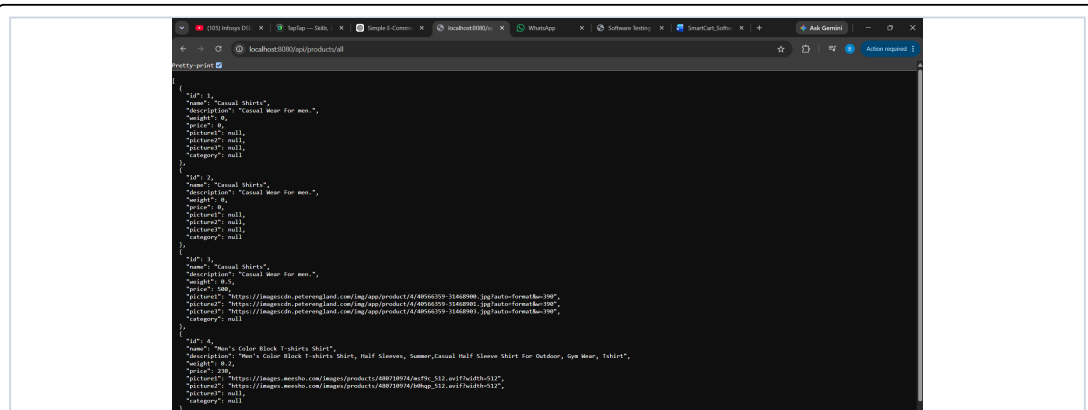


Figure 9.6: Backend Spring Boot Server REST API Response (Port 8080)

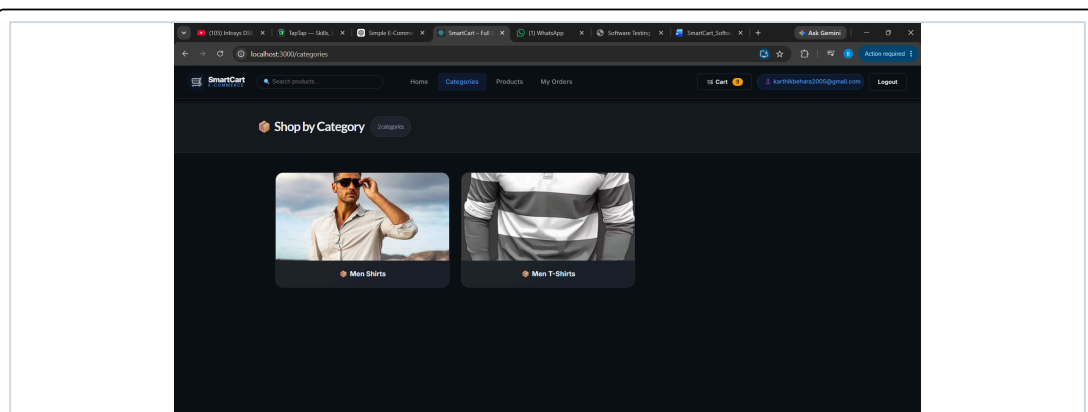


Figure 9.7: Frontend React Application Category View (Port 3000)

10. Conclusion

The **SmartCart – Full Stack E-Commerce Shopping Website** successfully demonstrates a complete, production-ready full-stack software application. By integrating a reactive React frontend with an enterprise Spring Boot REST API and a persistent MySQL relational database, the project fulfills all functional requirements, architectural standards, and quality criteria for major college submission.

SmartCart – Major Project Output Explanation